

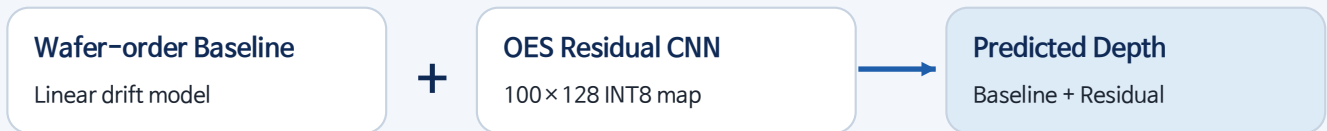
Zynq SoC 기반 Plasma Etch OES Virtual Metrology NPU

Wafer-order Baseline + Residual CNN 기반 INT8 NPU

BOSCH Plasma Etch 공정의 OES spectral-temporal 데이터로 Mean Si Etch Depth를 예측하고, Residual CNN inference를 Zynq FPGA의 INT8 NPU로 가속하는 HW/SW Co-design 프로젝트

DATASET 75 WAFERS 8 Lots / LOLO	TARGET WORKLOAD 1.39M MAC Residual Tiny CNN	HW TARGET SHARED 8-MAC Zynq PL / INT8
---	---	---

최종 예측 구조



현재 프로젝트 단계

Dataset / Sync / AI input	완료	Baseline / Model compare	완료
Residual CNN / Multi-seed	완료	INT8 PTQ / Fixed Golden	완료
FPGA RTL	이제 시작	AXI / Zynq integration	미구현
Vitis ARM SW	미구현	Real Zybo inference	미구현

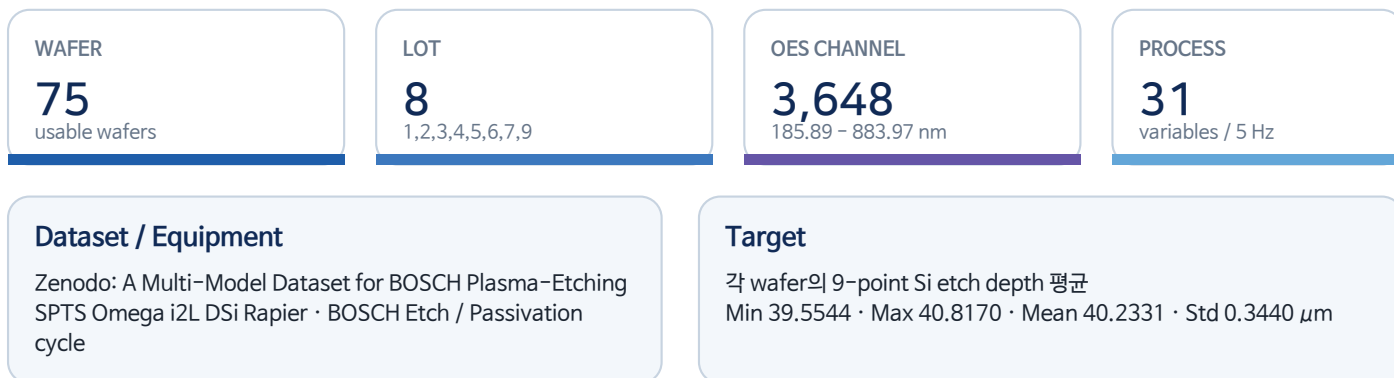
이번 회의의 핵심

AI 연구 단계에서 Digital Hardware Implementation 단계로 전환. NPU 구조, 메모리 규격, RTL 순서, PS/PL 인터페이스, 3인 역할을 동결한다.

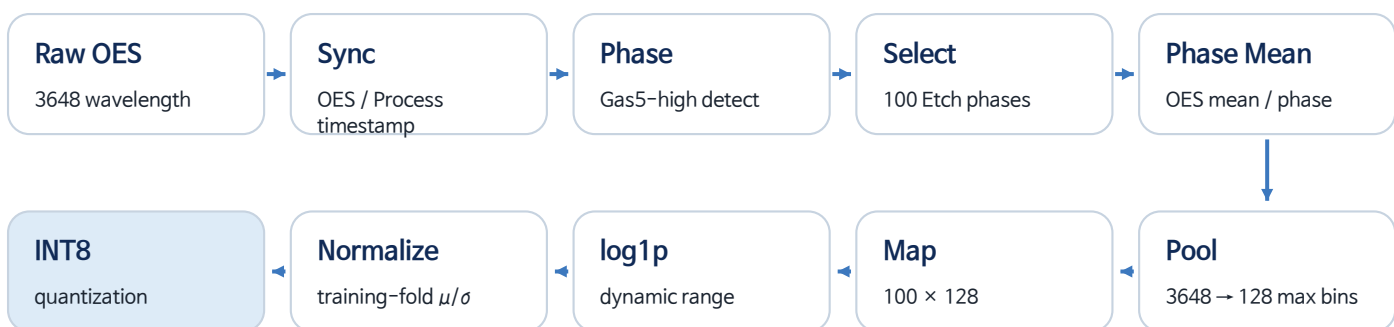
01 / DATA & PREPROCESSING

Dataset과 OES 전처리

75 usable wafers / 8 lots · 100 × 128 INT8 input



OES 전처리 Pipeline



FINAL CNN INPUT

100 × 128 × 1

12,800 INT8 values = 12.8 KB / wafer

전처리 검증

75 / 75 PASS · 실패 0건
 OES/Process time offset 약 0.10 – 0.25 s
 강한 correlation 확인

Gas5 chemistry 표기 주의

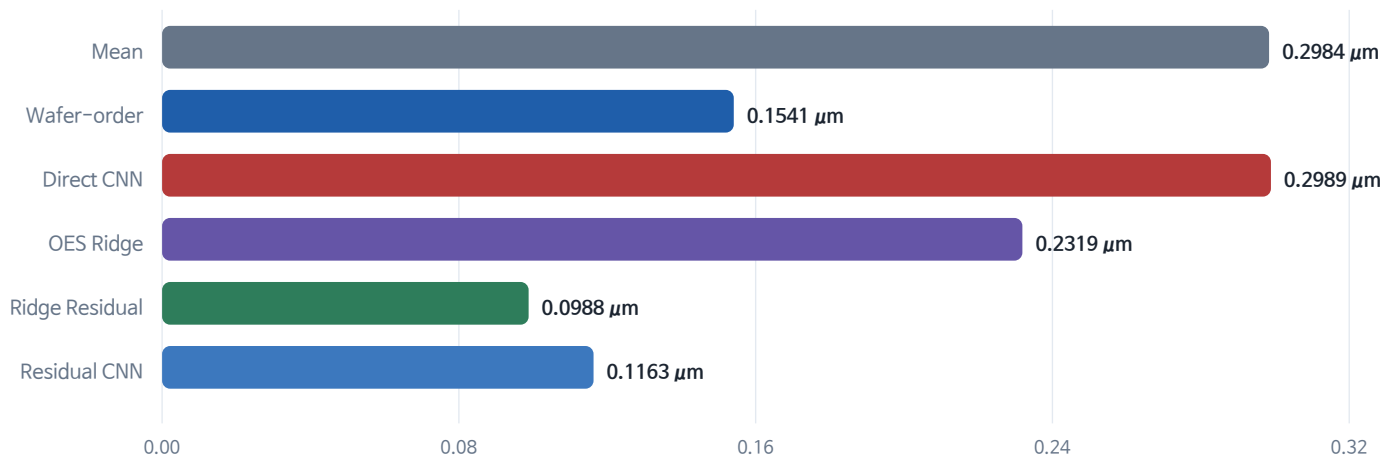
Gas5 feature timing이 약 4.5초 Etch 구간과 일치하여 Gas5 $\approx$ SF6 Etch phase로 강하게 추정한다. 단, feature 이름 자체에 chemistry mapping이 명시된 것은 아니므로 “Gas5가 SF6라고 공식 명시”라고 표현하지 않는다.

02 / MODEL SELECTION

AI 모델을 어떻게 결정했는가

Baseline 비교로 chamber drift를 확인하고 residual learning으로 재정의

LOLO MAE 비교 (낮을수록 좋음)



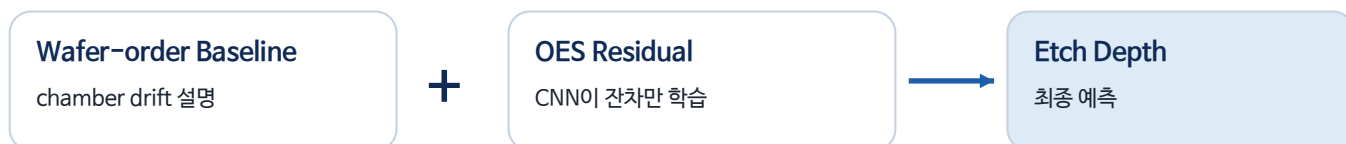
Direct Tiny CNN: 실패

OES → CNN → Absolute Depth
MAE 0.2989 μm으로 Mean predictor 수준. Prediction이 평균 값 근처로 collapse.

중요한 발견

Wafer 번호만 이용한 Linear Regression이 MAE 0.1541 μm.
Dataset에 wafer processing order에 따른 chamber drift가 강하게 존재.

문제 재정의: Baseline + Residual



모델 선택의 의미

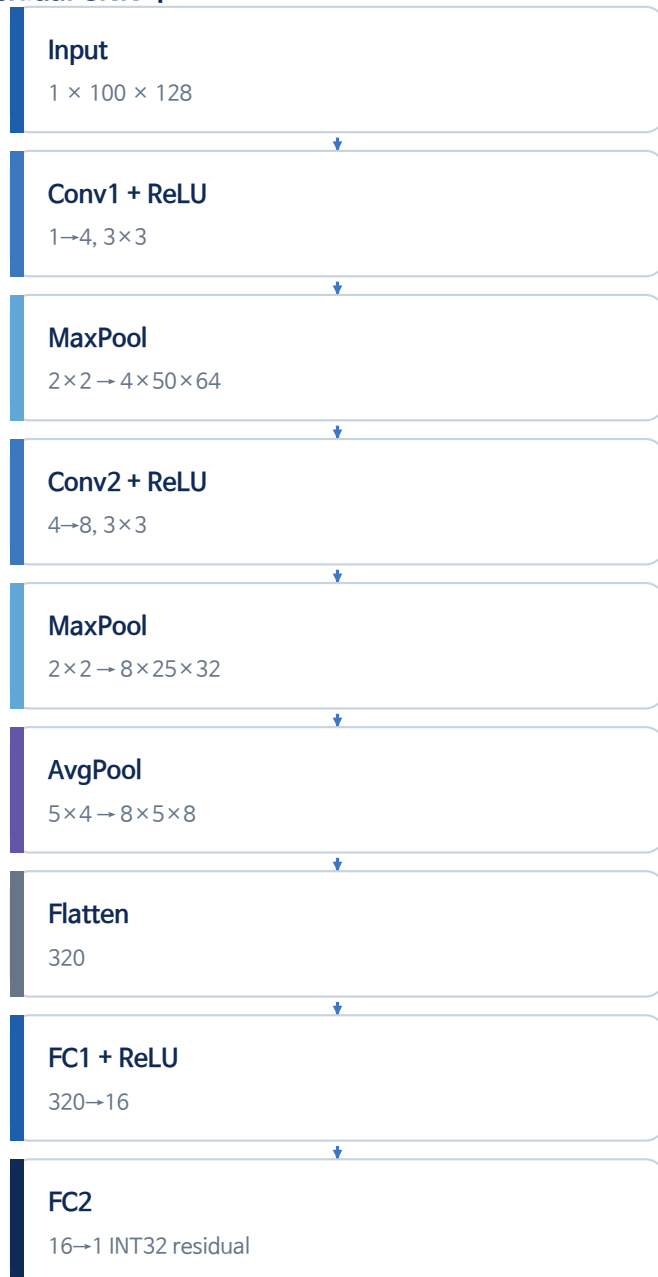
Ridge Residual MAE 0.0988 μm가 CNN보다 정확하다. CNN 선택 이유는 “가장 정확해서”가 아니다. Ridge는 hand-crafted OES Mean/Slope feature 기반 classical ML accuracy reference이고, CNN은 100×128 spectral-temporal map을 직접 처리하며 FPGA NPU 가속 대상 workload 다.

03 / CNN

Final Residual Tiny CNN

5,489 params · ~1,387,536 MAC / inference · INT8 weight ~5.46 KB

Residual CNN 구조



PARAMETERS

5,489
trainable

MAC

~1.39M
per inference

WEIGHT

~5.46 KB
INT8

DROPOUT

0.2
training only

Multi-seed Stability

Seeds 7 / 21 / 42 / 84 / 123
 5 seeds × 8 LOLO folds = 40 runs
 Mean MAE 0.1163 μm · Std 0.0051
 Best 0.1121 · Worst 0.1247 · Mean R² 0.8169

Lot-level note

Lot 5와 Lot 7의 개선 안정성이 상대적으로 약했다.

정확도 해석

Ridge Residual: MAE 0.0988 μm
 CNN multi-seed: Mean MAE 0.1163 μm

Accuracy만 보면 Ridge가 우수. CNN은 NPU workload로 채택.

04 / QUANTIZATION

INT8 / Fixed-point 준비 완료

RTL 정답 기준 = Python Fixed-point Golden Model

FP32 CNN MAE

0.1247 μm

LOLO seed 42

INT8 CNN MAE

0.1232 μm

LOLO seed 42

 R^2

0.8056

FP32 0.8020

FPGA 정수 연산 규격



Requantization (Conv1 / Conv2 / FC1)



Layer	M	Shift
Conv1	46,222	24
Conv2	139,797	24
FC1	224,171	24
FC2	requant 없음	-

FC2 Output

FC2는 requant하지 않는다.

FC2 INT32 Accumulator → PS ARM → Physical Residual [μm]Physical scale:
0.000014877117 μm / count

Fixed-point Golden 검증

Conv/Pool intermediate mismatch: 극소수 ± 1 LSB

FC1 output mismatch: 0 · FC2 accumulator mismatch: 0

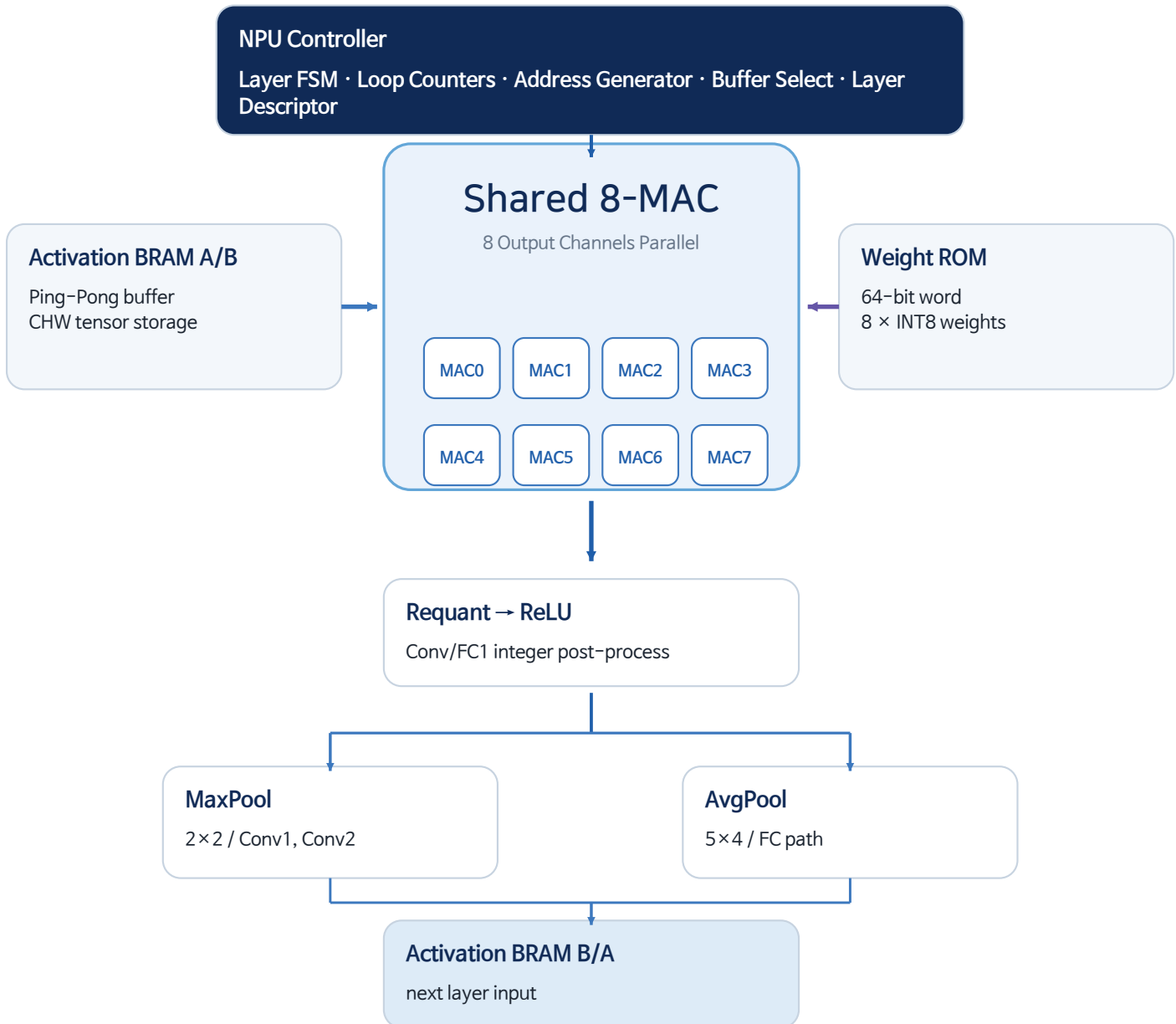
Final physical output difference: 0.0000000000 μm

핵심 파일: deployment_int8_model.npz · fixed_requant_params.npz · fixed_golden_result.npz · rtl_golden_sample0.npz

05 / NPU ARCHITECTURE

Shared 8-MAC NPU 아키텍처

Single engine reused by Conv1 → Conv2 → FC1 → FC2



8-MAC 병렬화: 8개의 Output Channel 병렬

Conv2는 $C_{in}=4$, $C_{out}=8$, $K=3 \times 3$. 한 output position에는 $4 \times 3 \times 3 = 36$ input terms 이 있다. 한 activation을 읽을 때 8개의 output-channel weight를 동시에 적용해 ACC0~ACC7을 갱신한다. 약 36 input term 처리 후 8개 output channel이 동시에 완성 된다.

Activation X

- W0 → ACC0
- W1 → ACC1
- W2 → ACC2
- W3 → ACC3
- W4 → ACC4
- W5 → ACC5
- W6 → ACC6
- W7 → ACC7

06 / MEMORY & CONTROL

Memory / Controller 설계

CHW · 32-bit activation packing · 64-bit weight packing · zero-injection padding

Activation BRAM A/B

Ping-Pong 구조

Memory layout = CHW

addr = channel × H × W + row × W + column

Packing: 4 × INT8 = 32-bit BRAM word

Weight Memory

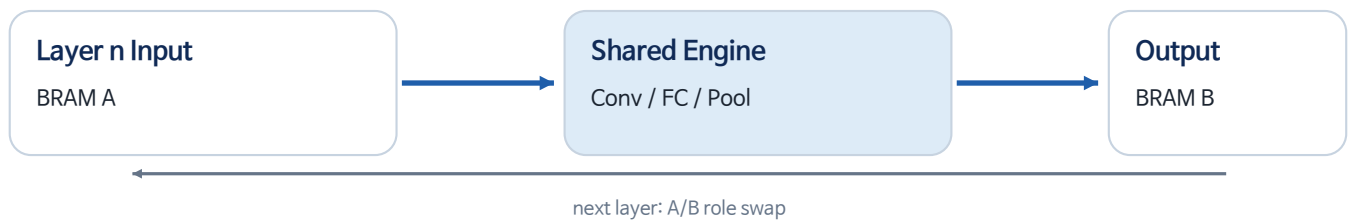
한 cycle에 8 output-channel weight 필요

8 × INT8 = 64-bit Weight Word

| W7 | W6 | W5 | W4 | W3 | W2 | W1 | W0 |

1차 버전: PL ROM/BRAM init data 고정

Ping-Pong 데이터 흐름



Padding = Zero Injection

별도의 padded tensor를 만들지 않는다.

Address Generator에서 row/col이 범위를 벗어나면

activation = 0 공급.

메모리 증가 없이 padding 처리.

Controller = Fixed Descriptor + FSM

완전 programmable microcode도 아니고 layer별 전용 datapath도 아님.

Generic Engine + Fixed Layer Descriptor로 동일 datapath 재사용.

Layer Descriptor 예시

Field	Layer 0 (Conv1)	Layer 2 (Conv2)
TYPE	CONV	CONV
H × W	100 × 128	50 × 64
Cin / Cout	1 / 4	4 / 8
K	3	3
M / Shift	46,222 / 24	139,797 / 24

07 / RTL IMPLEMENTATION

RTL 모듈 구성과 실제 구현 순서

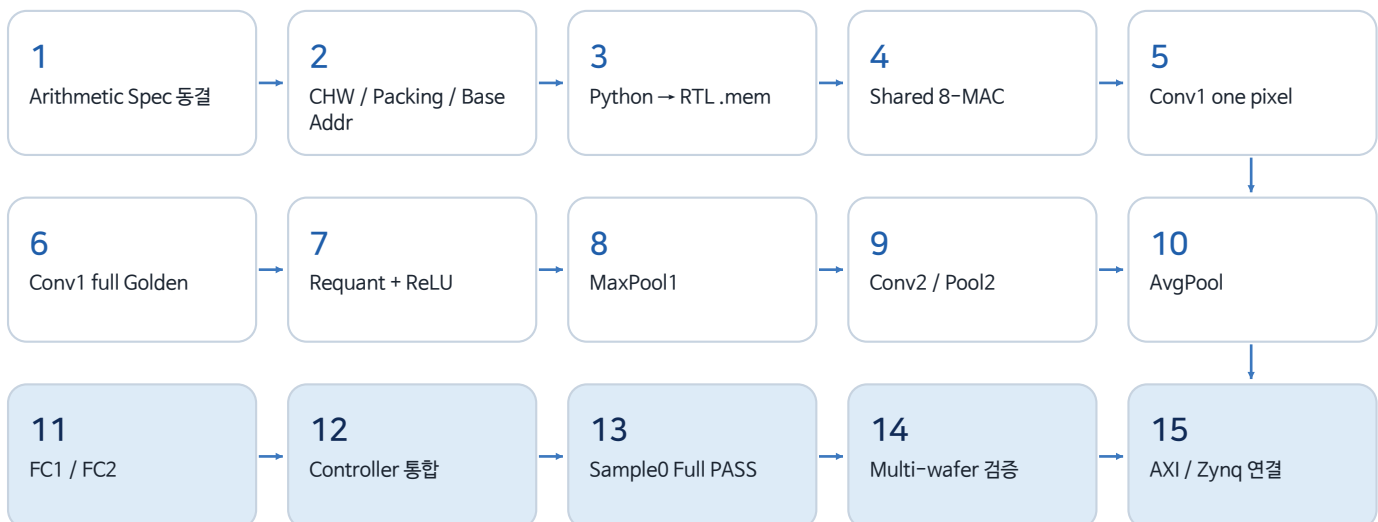
Core bit-exact first → AXI later

Module	Role
npu_core_top.v	순수 NPU Core Top
npu_controller.v	Layer FSM / Loop Counter / Buffer Select
address_generator.v	Conv / FC / Pool address 생성
mac_array_8.v	Shared 8-MAC
requant_unit.v	M×ACC / rounding / shift / saturation
maxpool_unit.v	2×2 Max Pool
avgpool_unit.v	5×4 sum / 20
activation_bram_A/B	Ping-Pong activation storage
weight_rom / bias_rom	INT8 weight / INT32 bias

개발 원칙: AXI부터 만들지 않는다

먼저 npu_core_top을 clk / reset / start / done / result[31:0] 인터페이스만으로 구현하고 Python Golden과 bit-exact를 완성한 뒤 AXI를 붙인다.

실제 구현 순서



검증 방법

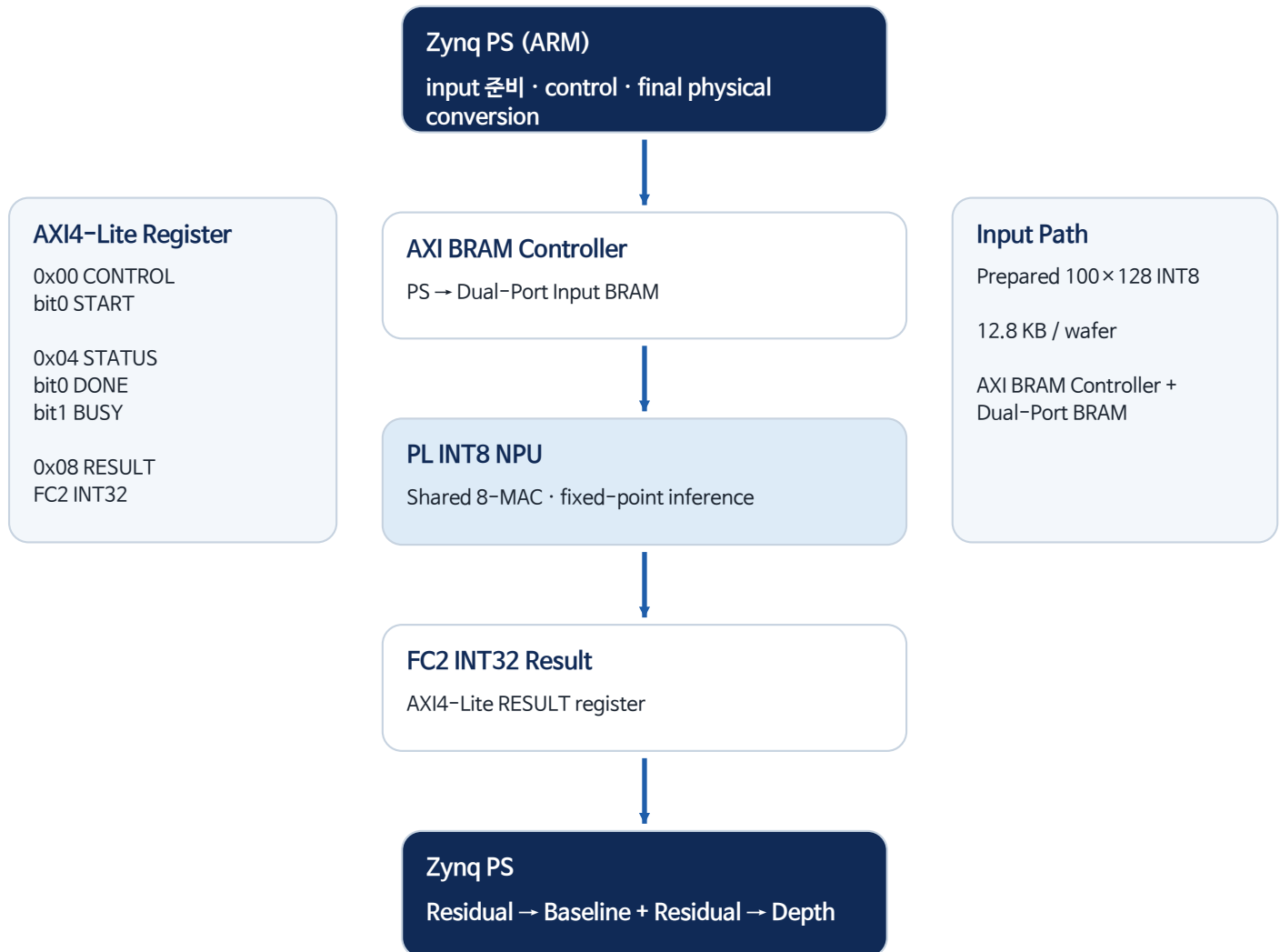
각 layer에서 RTL Output ↔ Python Golden을 비교한다. 최종 result 하나만 보지 않고 intermediate tensor까지 비교해야 주소 생성 / padding / requant / pooling 오류를 빠르게 분리할 수 있다.

08 / ZYNQ SOC

Vivado → AXI → Zynq PS/PL → Vitis

12.8 KB input은 AXI4-Lite register가 아니라 AXI BRAM Controller로 전달

최종 System Architecture



PS 계산

Deployment baseline:

$$\text{baseline} = -0.10232342 \times \text{wafer_order} + 40.77884674$$

Residual:

$$\text{residual} = \text{FC2_ACC} \times 0.000014877117 + \text{residual_mean}$$

Final:

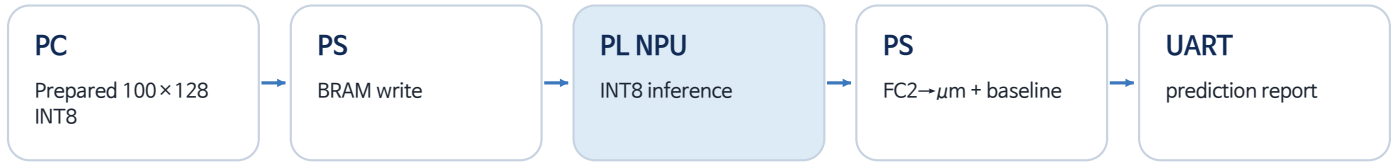
$$\text{Final Etch Depth} = \text{Baseline} + \text{Residual}$$

09 / DEMO & BENCHMARK

최종 시연 방법 및 Latency 측정

Prepared INT8 sample 기반 1차 demo · ARM SW vs FPGA NPU 실측

1차 시연 Data Flow



UART / 화면 출력

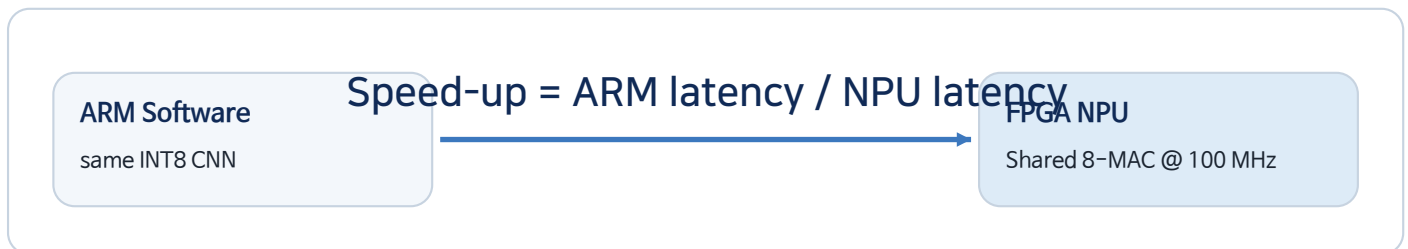
Lot / Wafer
Baseline
CNN Residual
Predicted Etch Depth
Measured Etch Depth
Absolute Error
NPU Latency

1차 Demo 범위

원본 NetCDF 전체 전처리를 ARM에서 하지 않는다.

PC에서 이미 생성한 100×128 INT8 sample을 사용해
PS→BRAM→PL→RESULT→PS end-to-end 경로부터 검증.

ARM SW vs FPGA NPU Latency



성능 수치 해석

현재 FPGA가 ARM보다 몇 배 빠를지는 확정되지 않았다. 8 MAC @ 100 MHz에서 순수 MAC만 계산한 이론적 하한은 약 1.73 ms지만 BRAM, 주소 생성, Requant, Pool, FSM overhead가 존재한다. Speed-up은 반드시 실제 보드 측정값으로 발표한다.

10 / TEAM EXECUTION

3인 역할 · 2주 일정 · 성공 기준 · 리스크

통합 interface를 Day 1에 합의하고 마지막 날 최초 통합을 피한다

팀원 A · NPU Datapath	팀원 B · Memory / Control	팀원 C · Zynq SoC / SW
- mac_array_8 - requant_unit - ReLU - maxpool / avgpool	- CHW / BRAM Packing - Address Generator - Padding / Ping-Pong - Layer FSM / Golden TB	- AXI4-Lite Wrapper - AXI BRAM Controller - Vivado BD / IP / XSA - Vitis C / UART / latency
CORE GOAL 정수 연산 규격을 RTL에서 정확히 구현	CORE GOAL Tensor dataflow와 bit-exact 검증	CORE GOAL 실제 Zybo end-to-end inference

통합 Interface 동결

A ↔ B: MAC input/output · valid · acc clear · weight order
 B ↔ C: BRAM address map · register map · START/DONE · RESULT format

약 2주 일정

D1	D2-3	D4	D5-6	D7	D8	D9	D10	Buffer
Spec	8-MAC+Conv1	Requant/Pool1	Conv2/FC	Full PASS	Multi-wafer/Synth	AXI/Vivado	Vitis/Zybo	Debug/Bench

성공 기준

1. Python Fixed Golden ↔ RTL Bit-Exact PASS
2. PS → BRAM → PL NPU → RESULT → PS on Zybo
3. 실제 Etch Depth Prediction · ARM SW vs FPGA NPU Latency
4. LUT / FF / BRAM / DSP Resource · Timing WNS

리스크 / 발표 원칙

Dataset: 75 wafers / 8 lots로 작음 · LOLO: 동일 BOSCH campaign 내부 unseen-lot feasibility estimate
 Ridge vs CNN: 0.0988 μm vs 0.1163 μm · Training MAE 0.0872는 일반화 성능으로 사용하지 않음

회의 최종 체크리스트

- ☐ 프로젝트 주제 = OES Virtual Metrology NPU
- ☐ 최종 예측 = Wafer-order Baseline + OES Residual CNN
- ☐ CNN / INT8 / Fixed Golden 변경 금지
- ☐ Shared 8-MAC · 8 Output Channel 병렬
- ☐ Activation CHW / 32-bit packing
- ☐ Weight 64-bit packing · 1차 PL ROM 고정
- ☐ Activation BRAM A/B Ping-Pong
- ☐ Padding = Address Generator Zero Injection
- ☐ Controller = Fixed Descriptor + FSM
- ☐ RTL Core Bit-Exact 완료 후 AXI 연결
- ☐ 첫 Demo = Preprocessed 100×128 INT8
- ☐ ARM SW vs FPGA NPU latency 실제 측정
- ☐ Ridge 우세 / Dataset 한계 투명 공개
- ☐ 3인 역할 및 통합 Interface 확정